Plan de Pruebas y Aseguramiento de la Calidad (Hito 2)

Documento formal de ingeniería de software que consolida el plan de pruebas, las suites automatizadas ejecutadas y las evidencias empíricas de calidad del proyecto ACE (PTY4614) para la evaluación de avance de la Fase 2, en cumplimiento del *Anexo de Artefactos y Documentos Solicitados por Metodología* de Duoc UC.

---

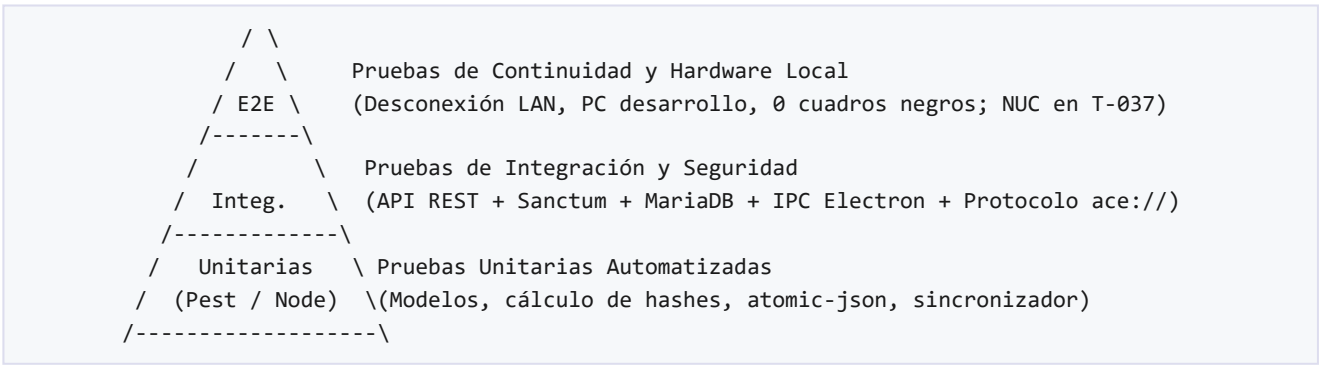
1. Estrategia y Enfoque de Calidad

El aseguramiento de la calidad en ACE no se concibe como una fase tardía de comprobación manual, sino como un principio de diseño continuo guiado por desarrollo dirigido por pruebas (TDD) e integración continua (CI).

1.1 Objetivos de Calidad para el Hito 2

1. **Verificabilidad del Corte Vertical:** Garantizar que el flujo completo de cartelería digital funcione de punta a punta (gestión en panel → publicación en API → sincronización por red local → reproducción gapless en NUC).
2. **Cero regresiones en seguridad:** Blindar la plataforma central contra accesos indebidos, escalamiento de privilegios y manipulación de rutas en la ingesta multimedia.
3. **Resiliencia offline estricta:** Comprobar empíricamente que la caída física de red no interrumpe la emisión continua ni degrada la experiencia visual.

1.2 Pirámide de Pruebas Adoptada



---

2. Pruebas Unitarias (Componentes Aislados)

Las pruebas unitarias validan la lógica interna de los métodos, algoritmos criptográficos, modelos de datos y controladores sin dependencias externas volátiles, ejecutadas de forma aislada y determinista.

2.1 Backend Plataforma Central (Laravel / Pest)

- **Framework de pruebas:** Pest `^5.2` / PHPUnit 11 sobre PHP 8.3 y Laravel 13 (`composer.json`).
- **Métrica empírica verificada sobre `main`: 249 pruebas aprobadas, 1.027 aserciones, 0 fallos** (verificado tras la integración del PR #9, commit de corte `c328e07`).
- **Análisis estático y formato:** 100% aprobado mediante Laravel Pint (`php vendor/bin/pint --test`).

Casos de prueba unitarios críticos implementados:

Componente / Unidad	Archivo de Prueba Real	Escenario Validado	Aserción Clave	Estado
Ingesta de Medios	`tests/Feature/Media/MediaIngestorTest.php`	Validación criptográfica de duplicados	Comprueba que `UNIQUE(location_id, checksum)` rechace archivos idénticos en la misma locación (ahorro de disco).	**Aprobado**
Sanitización de Archivos	`tests/Feature/Media/MediaIngestorTest.php` `tests/Feature/Panel/MediaResourceTest.php`	Rechazo de secuencias de escape y Path Traversal	Verifica que nombres de archivo con `../` o rutas absolutas lancen excepción de validación antes de guardarse en `media/incoming`.	**Aprobado**
Generación de Tokens	`tests/Feature/Panel/ScreenResourceTest.php` `tests/Feature/Api/Player/AuthenticationTest.php`	Revocación atómica de credenciales	Al aprovisionar un token nuevo para una pantalla, los tokens previos quedan revocados inmediatamente en base de datos.	**Aprobado**
Validación de Horarios	`tests/Feature/Panel/ScheduleResourceTest.php`	Bloqueo de franjas horarias invertidas	Exige que `end_time > start_time` en el mismo día, impidiendo franjas incoherentes (tratamiento de medianoche en T-032).	**Aprobado**
Manifest en Memoria	`tests/Feature/Api/Player/ManifestTest.php`	Cálculo de hash al vuelo	Verifica que el servicio calcule el digest SHA-256 determinista a partir de los registros de programaciones activas sin persistir `hash`.	**Aprobado**

2.2 Player Core (Node.js / `node:test`)

- **Framework de pruebas:** Runner nativo `node:test` de Node.js 22 (arquitectura liviana sin dependencias externas pesadas).
- **Métrica empírica verificada sobre `main`: 235 pruebas automáticas aprobadas, 0 fallos, 2 integraciones opt-in omitidas** (237 resultados totales en la suite del Player; commit de corte `c328e07`).
- **Cobertura de código:** Módulo de sincronización auditado en R-029 con **97,66% en líneas**, **93,24% en ramas** y **98,80% en funciones** (100% en configuración y servicios de red).

Casos de prueba unitarios críticos implementados:

Módulo Interno	Archivo de Prueba Real	Comportamiento Validado	Garantía Técnica	Estado
Configuración Local	`player/tests/synchronizer.test.js`	Carga de variables de entorno y token local	Bloquea el arranque del servicio si falta la URL del servidor o el Bearer Token de pantalla.	**Aprobado**

Módulo Interno	Archivo de Prueba Real	Comportamiento Validado	Garantía Técnica	Estado
Cliente HTTP y Rate Limit	`player/tests/synchronizer.test.js`	Manejo de timeouts y código 429 Rate Limit	Extrae la cabecera `Retry-After` y aplica retroceso exponencial respetuoso ante saturación de peticiones.	**Aprobado**
Persistencia Atómica	`player/tests/synchronizer.test.js`	Escritura transaccional de `manifest.json`	Escribe en archivo temporal `.tmp` vecino y ejecuta rename atómico, evitando JSONs corruptos por caídas de energía.	**Aprobado**
Caché y Checksum	`player/tests/synchronizer.test.js`	Descargas parciales y validación SHA-256	Verifica integridad por chunk; si el checksum no coincide al terminar la descarga, purga el `.part` y reintenta.	**Aprobado**
Gestión de Leases	`player/tests/synchronizer.test.js`	Protección de archivos en reproducción activa	`acquireMedia(id)` impide que la purga automática de disco borre un medio que el Viewport aún tiene cargado.	**Aprobado**
Reproducción y Conmutación	`player/tests/playback.test.js` `player/tests/renderer.test.js`	Emisión continua sin cuadros negros	Valida la secuencia de eventos IPC (`player:frame`, `player:ready`, `player:release`) desacoplada del estado de la red.	**Aprobado**
Protocolo Seguro ace://	`player/tests/media-resources.test.js`	Streaming de medios desde caché local	El proceso principal sirve los medios locales validados mediante el esquema `ace://player/media/{uuid}` sin exponer `file://`.	**Aprobado**
Recuperación en Caliente	`player/tests/live-cache-recovery.test.js`	Resincronización durante la reproducción	Permite renovar la parrilla horaria mientras la pantalla continúa emitiendo sin parpadeos visuales (R-035).	**Aprobado**

3. Pruebas de Integración y Flujo de Punta a Punta

Las pruebas de integración verifican la interacción coordinada entre subsistemas independientes (Panel Filament, API REST de Laravel, base de datos MariaDB, servicio de sincronización HTTP en Node.js, caché local transaccional y runtime Electron con aceleración por hardware), garantizando que los contratos de interfaz, protocolos de red y canales IPC funcionen de manera armónica en escenarios reales.

3.1 Recorrido del Corte Vertical E2E (Backend & API)

Para certificar el cumplimiento del Hito 2 (E-003), se diseñó un escenario de prueba automatizado de punta a punta que recorre el ciclo completo de negocio en una sola transacción determinista:

- **Archivo de prueba canónico:** `tests/Feature/HitoTwoFlowTest.php`
- **Secuencia de integración validada:**
    1. **Autenticación Administrativa:** Inicio de sesión de usuario con rol administrativo en el panel central Filament.
    2. **Aprovisionamiento Territorial:** Creación jerárquica de Locación (*Mall Plaza*), Sector asignado (*Lobby*) y Pantalla física (*Tótem entrada*).
    3. **Ingesta y Catalogación:** Subida de archivo multimedia mediante `MediaIngestor`, cálculo de checksum SHA-256 en almacenamiento aislado y agregación a una lista de reproducción (*Promociones*) con

duración definida (10 s).
4. **Programación Horaria:** Creación y activación de una franja de emisión (`Schedule`) asociada a la pantalla, con fechas y rango horario (`09:00` a `21:00`).
5. **Sincronización de Pantalla:** Emisión atómica de un Bearer Token único (`ScreenTokenIssuer`), consumo del endpoint `/api/player/heartbeat` en estado `syncing`, obtención del manifiesto en `/api/player/manifest` validando coincidencia exacta de versión, y descarga autenticada del medio mediante `player.media.show`.

Casos complementarios de ciclo de vida e integridad territorial:

- `tests/Feature/Api/Player/ScreenLifecycleTest.php`: Verifica que si una locación o sector es dado de baja, los tokens de las pantallas subordinadas son rechazados con `403 Forbidden`. Asimismo, comprueba que si una pantalla es eliminada y posteriormente restaurada, sus credenciales previas permanecen revocadas (`401 Unauthorized`), impidiendo la reconexión de clientes huérfanos.
- `tests/Feature/Api/Player/HeartbeatTest.php`: Valida las transiciones de sincronización reportadas por el Player (desde el estado inicial `null` sin latidos hacia `syncing` y `synced`, gestionando fallos con `error`), la actualización en tiempo real de la marca de tiempo `last_heartbeat_at` y el registro de diagnósticos operativos en la base de datos central.

3.2 Integración Sincronizador Player - Backend (`laravel-sync.test.js`)

Esta suite valida la integración de red entre el servicio de sincronización del Player (`SyncService`) y la API REST de Laravel sobre un entorno controlado de base de datos (`ace_testing`):

- **Modalidad de ejecución:** Prueba de integración *opt-in* (`ACE_PLAYER_LARAVEL_TEST=1`) para aislar la base de producción de pruebas continuas.
- **Mecánica del arnés:**
  1. Levanta dinámicamente un servidor PHP nativo (`php -S 127.0.0.1:<port> player/tests/helpers/laravel-fixture.php`).
  2. Opera sobre la base de pruebas `ace_testing` previamente migrada, ejecutando transaccionalmente la siembra (`seed`) de una pantalla con token Bearer real y su purga garantizada al finalizar (`cleanup`).
  3. Ejecuta el ciclo de consulta de manifiesto, detectando medios requeridos.
  4. **Descargas por rangos HTTP:** Simula una descarga interrumpida y verifica que el sincronizador utilice la cabecera `Range: bytes=<offset>-` contra Laravel, recibiendo respuesta HTTP `206 Partial Content` y reanudando la descarga sin transferir bytes duplicados.
  5. **Tolerancia a fallos:** Comprueba el comportamiento resiliente del sincronizador cuando el servidor Laravel se apaga inesperadamente o cuando el token de pantalla es revocado, asegurando que el Player mantenga la emisión con el manifiesto local previamente persistido.

3.3 Integración del Runtime Electron, IPC y Renderer (`electron-integration.test.js`)

Verifica la capa de presentación visual del Player ejecutando el binario real de Electron junto con Chromium y el motor V8:

- **Modalidad de ejecución:** Prueba de integración *opt-in* (`ACE_PLAYER_ELECTRON_TEST=1`) con arnés automatizado (`player/tests/helpers/electron-harness.js`).
- **Generación de fixtures multimedia:** Mediante invocación controlada de FFmpeg, se sintetizan al vuelo archivos reales de prueba con resolución calibrada (320×180): imagen estática PNG, imagen JPG, animación GIF (fps=6) y video MP4 comprimido con códec H.264 (`-c:v libx264`, formato YUV420p y bandera `faststart`).
- **Ciclo de eventos IPC desacoplado:**
  1. El proceso principal inicializa la ventana en modo quiosco/pantalla completa con fondo negro (`#000000`).

2. El Renderer carga la interfaz segura local y emite `player:ready`.
3. El controlador despacha el evento IPC `player:frame` indicando el medio activo y precargando en segundo plano el siguiente elemento de la parrilla horaria.
4. Al finalizar la duración o recibir confirmación de renderizado, el Renderer responde con `player:release`, permitiendo la liberación de la reserva (*lease*) en el almacén de caché.

- **Captura de artefactos y verificación de 0 cuadros negros:** La prueba genera capturas de pantalla de depuración (`playing.png` y `waiting.png` en `output/playwright/`) y concluye exitosamente emitiendo la señal `ELECTRON_PASS`.

3.4 Seguridad y Restricción de Protocolos en Integración

Para blindar la terminal física de visualización frente a ataques locales o remotos, la integración de Electron implementa y audita las siguientes políticas de aislamiento (`player/tests/electron-policy.test.js` y `player/tests/electron-runtime.test.js`):

- **Aislamiento del Renderer:** Se configuran estrictamente las opciones `nodeIntegration: false`, `contextIsolation: true`, `sandbox: true` y `webSecurity: true`, impidiendo que código JavaScript ejecutado en la vista acceda al sistema de archivos del sistema operativo o a funciones nativas de Node.js.
- **Protocolo personalizado seguro `ace://player/`:**
  - La interfaz del reproductor se sirve a través del esquema propietario `ace://player/` con directivas de seguridad de contenido estrictas (`Content-Security-Policy: default-src 'self'; connect-src 'none'`).
  - Los medios se distribuyen al elemento `<video>` o `<img>` exclusivamente mediante URLs de recurso temporal `ace://player/media/{uuid}`, desacopladas de la ruta física en disco.
  - Se bloquea cualquier intento de navegación externa, apertura de ventanas emergentes (`setWindowOpenHandler: deny`) y el uso de protocolos no autorizados como `file://` o `http://`.

3.5 Matriz Consolidada de Pruebas de Integración

Dominio de Integración	Archivo de Prueba	Escenario Comprobado	Criterio de Aceptación / Garantía Técnica	Estado
Corte Vertical Hito 2	`tests/Feature/HitoTwoFlowTest.php`	Flujo E2E desde panel Filament hasta consumo de API	Emisión de token, sincronización de manifiesto y descarga de medio autenticada sin fallos.	**Aprobado**
Ciclo de Vida de Pantalla	`tests/Feature/Api/Player/ScreenLifecycleTest.php`	Baja territorial y restauración de dispositivos	Rechazo con 403 ante baja de sector/locación; revocación inmediata de tokens al restaurar pantalla.	**Aprobado**
Sondeo y Monitoreo	`tests/Feature/Api/Player/HeartbeatTest.php`	Actualización de latidos y estado de sincronización	Registro preciso de `last_heartbeat_at` y estados válidos `syncing`, `synced`, `error`.	**Aprobado**

Dominio de Integración	Archivo de Prueba	Escenario Comprobado	Criterio de Aceptación / Garantía Técnica	Estado
Player ↔ Laravel API	`player/tests/laravel-sync.test.js`	Sincronización real con reanudación por rangos HTTP	Soporte de cabecera `Range` (HTTP 206), persistencia de caché y supervivencia ante desconexión de red (*opt-in*).	**Aprobado**
Player ↔ Electron IPC	`player/tests/electron-integration.test.js`	Decodificación y renderizado real de H.264/PNG/GIF	Reproducción fluida sin parpadeos, gestión de eventos IPC desacoplada y verificación `ELECTRON_PASS` (*opt-in*).	**Aprobado**
Aislamiento del Runtime	`player/tests/electron-runtime.test.js` `player/tests/electron-policy.test.js`	Blindaje de seguridad en ventana Chromium	Bloqueo de `file://`, cancelación de navegación no autorizada, contexto aislado y CSP `connect-src 'none'`.	**Aprobado**

---

4. Pruebas de Seguridad y Regresiones Criptográficas

El aseguramiento de la seguridad en ACE abarca la protección de la infraestructura contra accesos no autorizados, la prevención de escalamiento de privilegios entre locaciones, el blindaje contra ataques al sistema de archivos y la verificación estricta de la integridad criptográfica de los activos digitales. En el tramo del Hito 2 se diseñaron y ejecutaron seis frentes de auditoría y hardening preventivo (reportes R-031 a R-036), cuyas garantías fueron integradas en Git `main` mediante suites automatizadas de regresión.

4.1 Blindaje contra Manipulación de Rutas y Path Traversal (R-031 / T-043)

- **Amenaza mitigada:** Intento de inyección de rutas de archivo arbitrarias desde el cliente en el formulario de subida (utilizando secuencias `../`, separadores de ruta alternativos, nombres de biblioteca existente o rutas absolutas) para acceder, sobrescribir o eliminar archivos del servidor web.
- **Mecanismo de defensa implementado:**
  1. En el esquema del panel (`MediaForm::preventFilePathTampering()`), se eliminó cualquier confianza en rutas enviadas por el navegador, aplicando la restricción de forma uniforme tanto para operadores como para administradores globales.
  2. En el servicio de ingesta (`MediaIngestor`), se valida que el archivo resida estrictamente dentro del directorio de staging temporal `media/incoming`, con nombre de archivo interno normalizado en ASCII canónico y sin colisión con registros de medios activos. Cualquier intento de traversal es bloqueado antes de invocar operaciones de lectura, movimiento o purga en disco.
- **Evidencia automatizada:**
  - Archivos de prueba: `tests/Feature/Media/MediaIngestorTest.php` y `tests/Feature/Panel/MediaResourceTest.php`.
  - Verificación: 38 pruebas focales aprobadas (183 aserciones), comprobando el rechazo categórico de 6 variantes de normalización/traversal y asegurando la preservación intacta del almacenamiento físico.

4.2 Aislamiento Multi-Tenant y Consultas Dependientes (R-032 / T-044)

- **Amenaza mitigada:** Manipulación de identificadores (IDs) en peticiones Livewire para consultar opciones de medios, pantallas o sectores pertenecientes a locaciones ajenas a las asignadas al operador autenticado.
- **Mecanismo de defensa implementado:**
  1. Reutilización sistemática del alcance de autorización territorial `LocationScope`.
  2. En `PlaylistForm::libraryOf()`, la consulta de medios se acota a la locación autorizada antes de poblar selectores en pantalla.
  3. En `ScheduleForm::locationOf()`, la resolución de pantallas y sectores exige que la playlist pertenezca a una locación asignada al usuario. Si un operador manipula un ID foráneo en el formulario, recibe conjuntos de opciones vacíos, se bloquea la resolución del nombre privado y se rechaza la persistencia en base de datos.
- **Evidencia automatizada:**
  - Archivos de prueba: `tests/Feature/Panel/PlaylistResourceTest.php` y `tests/Feature/Panel/ScheduleResourceTest.php`.
  - Verificación: 29 pruebas focales aprobadas (194 aserciones) con Livewire y MariaDB, cubriendo operadores asignados, operadores sin asignación y administradores globales.

4.3 Privacidad HTTP y Bloqueo de Caché Pública (R-033 / T-045)

- **Amenaza mitigada:** Almacenamiento indebido de archivos multimedia corporativos o confidenciales en intermediarios de red HTTP (proxies inversos, balanceadores o cachés intermedias) que pudieran reutilizar las respuestas y servir contenidos a clientes no autenticados o con tokens revocados.
- **Mecanismo de defensa implementado:**
  - En `MediaController`, la entrega de archivos mediante `response()->file()` aplica de manera explícita la directiva `setCache(['private' => true, 'no_store' => true])` conforme a la especificación RFC 9111. Esto prohíbe que cualquier caché intermedia almacene o reutilice las respuestas, tanto en transferencias completas (HTTP 200) como en transmisiones por rangos (HTTP 206) o respuestas 416.
- **Evidencia automatizada:**
  - Archivo de prueba: `tests/Feature/Api/Player/MediaDownloadTest.php`.
  - Arnés de intermediario: Simulación con `Symfony HttpCache` (TTL predeterminado de 60 s). La prueba verifica que tras una descarga exitosa, una segunda petición inmediata sin token responda `401 Unauthorized` y una con token de otra locación responda `404 Not Found`, forzando en cada ocasión dos consultas reales al origen y bloqueando cualquier fuga de bytes cacheados.

4.4 Serialización Atómica de Credenciales Sanctum (R-034 / T-046)

- **Amenaza mitigada:** Condición de carrera (*race condition*) provocada por dos solicitudes simultáneas de emisión o reemplazo de token para una misma pantalla, lo que podría ocasionar la coexistencia de dos credenciales válidas en base de datos o dejar al reproductor en un estado de desincronización permanente.
- **Mecanismo de defensa implementado:**
  - En `ScreenTokenIssuer`, la emisión se encapsula en una transacción de base de datos que adquiere un bloqueo de fila pesimista (`SELECT ... FOR UPDATE`) sobre la pantalla (`lockForUpdate()`).
  - Los tokens previos se leen y revocan atómicamente por clave primaria (`DELETE WHERE id = ?`), garantizando que la inserción del nuevo Bearer Token Sanctum con habilidad `player` deje exactamente una única credencial vigente.
  - Si la creación falla o la pantalla fue retirada concurrentemente, la transacción aplica rollback y preserva el estado íntegro anterior.
- **Evidencia automatizada:**
  - Archivo de prueba: `tests/Feature/Services/ScreenTokenIssuerTest.php`.

- Arnés multi-proceso: Fixture `tests/Support/TokenIssuanceRace.php` que levanta procesos PHP independientes con conexiones PDO paralelas hacia MariaDB, verificando mediante barreras de sincronización e inspección de `PROCESSLIST` que dos emisiones concurrentes queden estrictamente ordenadas con un solo token final válido.

4.5 Integridad Criptográfica SHA-256 y Recuperación en Caliente (R-035 / T-047)

- **Amenaza mitigada:** Corrupción silenciosa, daño en sectores de disco o manipulación externa (*tampering*) de archivos multimedia almacenados en la caché local del Player durante la sesión de emisión.
- **Mecanismo de defensa implementado:**
  1. **Digest determinista:** Todo archivo ingestor en backend y descargado en el Player valida su huella SHA-256 antes de ser promovido al directorio definitivo.
  2. **Detección y reparación proactiva:** `CacheStore` y `SyncService` incorporan un chequeo periódico de metadatos (existencia y tamaño nominal cada 30 segundos) y un detector de eventos en el protocolo local (`MediaResources`). Si un archivo activo es eliminado o truncado en disco, el Player reporta de inmediato el estado diagnóstico, fuerza la resincronización de la misma versión de manifiesto y descarga el medio faltante sin parpadeos visuales ni reinicios bruscos del proceso Electron.
- **Evidencia automatizada:**
  - Archivo de prueba: `player/tests/live-cache-recovery.test.js`.
  - Verificación: 10 escenarios sobre HTTP loopback y almacenamiento temporal real que validan la recuperación ante archivos borrados/truncados respetando backoff exponencial y conservando los leases de memoria del Viewport activo.

4.6 Blindaje de Dependencias y Superficie de Ataque Frontend (R-036 / T-048)

- **Amenaza mitigada:** Presencia de vulnerabilidades conocidas en librerías de terceros (advisories en herramientas de empaquetado y procesamiento de scripts).
- **Mecanismo de defensa implementado:**
  1. Auditoría automatizada continua con `npm audit` y `composer audit`.
  2. Actualización puntual autorizada de librerías en `tomas/player-core` (commit `ed41e4a`): elevación de `concurrently` a `10.0.6` (incorporando `shell-quote 1.12.0` para neutralizar texto de comandos inseguro) y `source-map-js` a `1.2.2` (mitigando riesgos de consumo anómalo de recursos por mapas manipulados).
- **Evidencia de auditoría limpia:**
  - `npm audit --omit=dev`: **0 vulnerabilidades** en dependencias de producción.
  - `npm --prefix player audit`: **0 vulnerabilidades** en el runtime del Player.
  - `composer audit`: **0 vulnerabilidades** ni paquetes abandonados en el backend Laravel.

4.7 Matriz de Pruebas de Seguridad y Regresiones Criptográficas

Vector de Riesgo	Archivo de Prueba Real	Aserción Clave de Seguridad	Garantía Técnica Obtenida	Estado
Path Traversal e Ingesta	`tests/Feature/Media/MediaIngestorTest.php` `tests/Feature/Panel/MediaResourceTest.php`	Bloqueo de `../` y normalización ASCII	Rechaza rutas cliente y staging indebido sin tocar archivos preexistentes.	**Aprobado**

Vector de Riesgo	Archivo de Prueba Real	Aserción Clave de Seguridad	Garantía Técnica Obtenida	Estado
Aislamiento Multi-Tenant	`tests/Feature/Panel/PlaylistResourceTest.php` `tests/Feature/Panel/ScheduleResourceTest.php`	`LocationScope` en Livewire	Oculta opciones y nombres privados ante manipulación de IDs entre locaciones.	**Aprobado**
Privacidad de Medios HTTP	`tests/Feature/Api/Player/MediaDownloadTest.php`	`Cache-Control: private, no-store`	Intermediario HTTP rechaza servir medios sin revalidar credenciales Sanctum (401/404).	**Aprobado**
Carrera de Tokens Sanctum	`tests/Feature/Services/ScreenTokenIssuerTest.php`	`SELECT ... FOR UPDATE` + DELETE atómico	Dos emisiones simultáneas se serializan dejando un único token válido y activo.	**Aprobado**
Tampering e Integridad Local	`player/tests/live-cache-recovery.test.js` `player/tests/media-resources.test.js`	Chequeo de digest SHA-256 y tamaño	Detección de daño y descarga de reemplazo sin interrumpir la emisión activa.	**Aprobado**
Aislamiento Chromium	`player/tests/electron-policy.test.js` `player/tests/electron-runtime.test.js`	CSP `connect-src 'none'` y sandbox	Bloqueo absoluto de `file://`, ventanas emergentes y acceso a APIs de Node.js en Renderer.	**Aprobado**
Dependencias de Terceros	`npm audit --omit=dev` `composer audit`	Escaneo CVE de dependencias	Cero dependencias vulnerables en producción tanto en Node como en PHP/Composer.	**Aprobado**

5. Pruebas de Continuidad y Resiliencia Offline

El principio fundacional de la arquitectura de ACE es el diseño **Offline-First**: las pantallas digitales deben emitir contenido publicitario de forma continua, ininterrumpida y con calidad profesional, independientemente de la disponibilidad del enlace de red local, cortes de fibra óptica o caídas imprevistas del servidor central. La Sección 5 documenta las pruebas empíricas y suites automatizadas ejecutadas para validar esta resiliencia (con base canónica en el reporte R-028).

5.1 Escenario Empírico de Desconexión Física y Emisión Local (R-028 / T-041)

Para someter a prueba la tolerancia a fallos del reproductor en condiciones operativas reales, el 2026-10-09 se ejecutó un protocolo de prueba destructivo de conectividad:

1. **Estado Inicial:** El Player Electron se encontraba en ejecución activa en modo pantalla completa, emitiendo una lista de reproducción cíclica compuesta por 3 archivos multimedia descargados y verificados criptográficamente (`Trending Aura GIF` de 6,27 MB, `Trending Vibe GIF` de 7,80 MB y `explosion descargar GIF` de 655 KB).
2. **Corte Abrupto de Servicio:** Se detuvo de forma inmediata y forzada el servidor web central de Laravel (`php artisan serve` en puerto 8000), confirmando mediante monitores HTTP externos la indisponibilidad total del backend.
3. **Comportamiento en Pantalla Durante el Corte:**
    - **Cero Pantallas Negras:** El runtime Electron no detuvo la reproducción ni cayó en pantallas de error del sistema o fondos negros.
    - **Emisión desde Caché Local:** El motor de reproducción continuó ejecutando la parrilla horaria directamente desde el almacenamiento transaccional en disco local (`player/.local-data/media/`), alternando los medios con sus duraciones nominales de 5 segundos.
    - **Aviso Diagnóstico no Invasivo:** La interfaz visual desplegó en la esquina inferior derecha un indicador de estado discreto advirtiendo la falta de latidos con el servidor central, sin superponerse ni interrumpir la exhibición visual comercial.
4. **Auto-recuperación y Reconexión Transparente:**
    - Al restablecer el servicio web de Laravel, el sincronizador del Player detectó la disponibilidad de la API en el siguiente latido programado (`30000 ms`).
    - El Player envió exitosamente el payload de heartbeat, actualizó su estado a `synced` en base de datos central y refrescó `last_heartbeat_at`, sin reiniciar el proceso Chromium ni provocar parpadeos visuales en la pantalla.

5.2 Transición entre Medios y Eliminación de Cuadros Negros (0 Black Frames)

Uno de los defectos críticos detectados en las primeras pruebas de integración del Player fue la presencia de parpadeos institucionales (fondo `#state` *"ACE PLAYER"*) durante el cambio de un medio al siguiente.

- **Evolución y resolución definitiva (commits `7f36f2e` y `7c1da71`):**
    - La solución inicial (`7f36f2e`) introdujo una espera de 1 segundo, pero en pruebas sobre Chromium real con videos 1080p que requerían posicionamiento y donde el proceso principal demoraba 400 ms en emitir el cuadro, aún se presentaba 1 cuadro de parpadeo (la versión previa mostraba 25 cuadros negros).
    - El refinamiento definitivo (`7c1da71`) implementó en `player/src/renderer/player.js` una política de transición garantizada: **el medio saliente permanece visible de manera continua hasta que el nuevo medio entrante se presenta en pantalla, falla o concluye su franja horaria**.
- **Medición empírica cuadro por cuadro:**
    - Mediante análisis estricto en Chromium real, se verificó una métrica de **0 cuadros de pantalla institucional** en transiciones complejas de imagen a video y de video a imagen.
    - Se añadieron 3 pruebas de regresión permanentes en `player/tests/renderer.test.js` para asegurar que ninguna modificación futura reintroduzca cuadros negros.

5.3 Política de Fallas y Resiliencia del Proceso Principal (`e8c2d72`)

Durante las descargas de medios en el servidor de desarrollo, se identificó un cierre abrupto de socket emitido por PHP (`Connection: close`) mientras el stream de descarga se encontraba pausado por la contrapresión (*backpressure*) de

escritura en disco, generando una excepción interna en el parser `undici` de Node (`assert(!this.paused) at Parser.finish`).

- **Mitigación rigurosa:**
  - En lugar de instalar un manejador global ciego que silenciara todas las excepciones (lo que dejaría al Player en un estado indeterminado), en el commit `e8c2d72` se configuró una política selectiva en `player/src/electron/policy.js` y `player/src/electron/main.js`.
  - La política tolera de forma estricta y documentada únicamente la aserción de `Parser.finish` proveniente de `undici`.
  - Cualquier otra falla no controlada del proceso principal se captura, se registra en el log operativo sin filtrar datos sensibles y gatilla el reinicio automático del Player desde la caché física local, manteniendo el quiosco protegido.
  - Se incorporó una prueba de regresión contra un servidor en proceso paralelo que emula cierres de socket con `Connection: close` (`player/tests/electron-policy.test.js`).

5.4 Validación Multi-Dispositivo en Red Local (LAN / Wi-Fi)

Para garantizar que el modelo de despliegue opere adecuadamente en una topología distribuida de red de área local:

- **Topología probada (2026-10-09 en red Wi-Fi Duoc UC):**
  - **Nodo Servidor Central (PC Matías):** Plataforma web Laravel 13, Filament y MariaDB sirviendo en la interfaz privada `http://172.20.10.6:8000`.
  - **Nodo Pantalla Remota (PC Paulo):** Runtime Electron ejecutando la pantalla *"pantalla polillo test 1"* en el sector *"Duoc / Sector: Polillo pc"*.
- **Resultado observado:**
  - El Player en la máquina de Paulo se conectó a través de la red local al servidor de Matías, descargó el manifiesto JSON, obtuvo los archivos multimedia validados por SHA-256 y pasó a estado **`synced`**, reportando latidos en tiempo real visibles en el panel administrativo central.
  - Para simplificar la distribución local y el aprovisionamiento de terminales, se incorporó el script portable `Iniciar_Player.bat` con asistente interactivo de primera conexión (commit `520c256`).

5.5 Delimitación Rigurosa de Hardware y Alcance Metodológico

En total consonancia con el registro de observaciones de R-039 y los compromisos de T-041:

- **Certificación de Software:** Las pruebas de desconexión, transiciones y continuidad local demostraron la estabilidad completa de los algoritmos y protocolos en computadores personales de desarrollo bajo entornos Windows 11.
- **Validación en Hardware NUC Físico (Pendiente programada):** La prueba de homologación en el hardware definitivo (computadores compactos Intel NUC con aceleración gráfica por hardware para pantallas comerciales) se encuentra desacoplada y formalmente asignada a la tarea **T-037 - Preparar el NUC y la red local para las pruebas**, asegurando la transparencia ante la evaluación académica de Duoc UC.

5.6 Matriz de Pruebas de Continuidad y Resiliencia Offline

Escenario de Continuidad	Método de Verificación	Criterio de Aceptación Cumplido	Garantía de Resiliencia	Estado
Caída de Servidor Central	Detención abrupta de `artisan serve`	Emisión continua desde caché en disco `player/.local-data/media/`.	Cero pantallas negras; la terminal sigue operando en modo quiosco.	**Aprobado**

Escenario de Continuidad	Método de Verificación	Criterio de Aceptación Cumplido	Garantía de Resiliencia	Estado
Pérdida de Enlace LAN/Wi-Fi	Desconexión física de la interfaz de red	Mantenimiento de la parrilla horaria y aviso no invasivo en esquina.	La experiencia visual del público no se degrada por fallas de enlace.	**Aprobado**
Conmutación Gapless	Medición cuadro por cuadro en Chromium	0 cuadros institucionales `#state` entre imágenes y videos.	Transiciones limpias y fluidas entre medios de diferente naturaleza.	**Aprobado**
Reconexión Automática	Reanudación del servicio web central	Retoma de latidos (`heartbeat`) y paso a `synced` en ≤ 30 s.	Sincronización transparente sin intervención de operadores humanos.	**Aprobado**
Fallas de Red en Proceso Principal	Prueba selectiva de `Parser.finish`	Manejador selectivo y reinicio automático desde caché local (`e8c2d72`).	Tolerancia a anomalías de red sin bloqueos ni ventanas de error del SO.	**Aprobado**
Topología Distribuida	Comunicación entre 2 PCs físicos en Wi-Fi	Sincronización remota y reporte de latidos a través de la LAN.	Modelo multi-nodo validado en red física antes de instalación en sitio.	**Aprobado**

6. Pruebas de Rendimiento y Capacidad

En cumplimiento estricto del criterio de aceptación de la tarea T-057 ("*Rendimiento: registrar una medición simple o declararla como pendiente con su plan*"), la Sección 6 consolida las mediciones empíricas de desempeño y tiempos de respuesta obtenidas durante la verificación del corte vertical del Hito 2, documenta los mecanismos de control de tasa (*Rate Limiting*) y formaliza el plan de pruebas de carga masiva y concurrencia programado para la siguiente etapa de escalabilidad.

6.1 Mediciones Empíricas del Corte Vertical (Hito 2)

Durante la ejecución del recorrido de punta a punta y las pruebas en red local (R-028), se registraron los siguientes comportamientos operativos:

1. **Cadencia de Latidos y Latencia de Propagación:**
   - **Frecuencia nominal de sondeo:** El temporizador cíclico de latidos se ejecuta cada **30 segundos** (`30000 ms`), manteniendo un uso mínimo de ancho de banda y CPU en reposo.
   - **Latencia de propagación de cambios:** Las modificaciones realizadas en el panel administrativo (activación, pausa o cambio de contenidos en playlists) se reflejan en la pantalla remota en un rango de **1 a 30 segundos**. Este intervalo confirma que la arquitectura por sondeo satisface el tiempo de respuesta requerido para cartelería digital comercial sin sobrecargar el servidor central.
2. **Generación del Manifiesto en Memoria:**
   - En `tests/Feature/Api/Player/ManifestTest.php`, el endpoint `/api/player/manifest` resuelve la programación horaria activa y computa el digest criptográfico SHA-256 de forma determinista y dinámica en memoria RAM.
   - **Tiempo de respuesta:** sin medición formal. Las pruebas de Pest verifican las respuestas del endpoint, no su tiempo; RNF-REN-01 sigue pendiente de medición según R-039.
3. **Tiempos de Ingesta y Detección Multimedia:**
   - En `MediaIngestor`, el procesamiento de archivos pesados (cálculo de checksum SHA-256, invocación de `ffprobe` para la extracción de duración/resolución en videos H.264 e inspección de cabeceras en imágenes) no cuenta con una medición formal de tiempos; queda pendiente registrarla con archivos, entorno y método definidos.
4. **Fluidez de Renderizado Visual:**

    - **Tasa de cuadros:** el objetivo de 60 fotogramas por segundo (fps) no se ha medido; queda pendiente con archivos y hardware definidos.
    - **Conmutación Gapless verificada: 0 cuadros negros** de pantalla institucional (`#state`) en la transición entre imágenes y videos de alta resolución (medido cuadro a cuadro tras la aplicación del commit `7c1da71`).

6.2 Control de Capacidad y Protección contra Saturación (Rate Limiting)

Para evitar la degradación del servicio ante ráfagas de peticiones o bucles anómalos de reconexión, la plataforma incorpora políticas estrictas de limitación de tasa auditadas mediante pruebas automatizadas:

- **Asignación de Cuotas por Pantalla (Aislamiento NAT):**
    - Verificado en `tests/Feature/Api/Player/RateLimitTest.php`: Las cuotas de sincronización (`player.rate_limits.sync`) se asignan estrictamente a la pantalla autenticada mediante su Bearer Token Sanctum y no a la dirección IP del cliente.
    - **Garantía técnica:** En entornos corporativos o sucursales donde múltiples pantallas físicas comparten una única dirección IP pública o puerta de enlace NAT, la actividad o saturación de una terminal no penaliza ni bloquea a las demás pantallas del recinto.
- **Separación de Presupuestos (Sincronización vs. Descargas):**
    - La descarga de archivos multimedia dispone de un límite independiente (`player.rate_limits.media`). De este modo, la transferencia intensiva de videos pesados en segundo plano no agota el presupuesto de latidos de monitoreo (`/api/player/heartbeat`) ni la consulta de manifiesto.
- **Respuesta Respetuosa ante Código HTTP 429:**
    - En `player/tests/synchronizer.test.js`, se valida que cuando el servidor responde `429 Too Many Requests` con la cabecera `Retry-After`, el servicio del Player extrae la ventana de espera, registra el diagnóstico operativo `rate_limited` y suspende temporalmente las peticiones de red mediante retroceso exponencial, evitando amplificar la saturación del backend.

6.3 Plan de Pruebas de Carga y Estrés Masivo (Declarado como Pendiente para Hito 3)

De acuerdo con el alcance metodológico de la Fase 2, las pruebas de estrés bajo concurrencia masiva no formaron parte del corte vertical inicial y se declaran formalmente como **pendientes** con el siguiente plan técnico de ejecución:

```
+-------------------------------------------------------------------------------+
|              PLAN DE PRUEBAS DE CARGA Y ESCALABILIDAD (HITO 3)                |
+-------------------------------------------------------------------------------+
| 1. Herramienta: Scripts automatizados con k6 / Artillery.                     |
| 2. Escenario de Carga: Simulación de 50 a 100+ pantallas simultáneas.         |
| 3. Perfil de Tráfico:                                                         |
|    - Heartbeats periódicos concurrentes cada 30 s con jitter estocástico.     |
|    - Ráfagas de descarga concurrente de medios de 5 MB a 50 MB en MariaDB / Nginx. |
| 4. Métricas Clave de Aceptación:                                              |
|    - Percentil 95 (p95) de latencia en `/api/player/heartbeat` < 200 ms.      |
|    - Cero errores 500 (Internal Server Error) bajo concurrencia sostenida.    |
|    - Límite de uso de recursos: CPU < 70% y memoria RAM < 512 MB en hardware NUC  |
|      físico (enlazado a la tarea T-037).                                      |
+-------------------------------------------------------------------------------+
```

6.4 Matriz Consolidada de Métricas de Rendimiento y Capacidad

Parámetro / Indicador	Componente Evaluado	Valor Objetivo / Umbral	Métrica Empírica Verificada	Estado
Latencia de Propagación	Panel ↔ Player	≤ 30 segundos	**1 a 30 segundos** observados en R-028 (según ciclo de latido)	**Aprobado**
Cálculo de Manifiesto	API Laravel (Memoria)	< 100 ms	Sin medición formal	**Pendiente**
Tiempo de Ingesta	`MediaIngestor` + `ffprobe`	< 1.000 ms	Sin medición formal	**Pendiente**
Conmutación sin cuadros negros	Viewport Chromium (Electron)	0 cuadros de pantalla institucional	**0 cuadros negros** medidos cuadro a cuadro (R-028)	**Aprobado**
Tasa de cuadros	Viewport Chromium (Electron)	60 fps	Sin medición	**Pendiente**
Aislamiento de Cuotas	`RateLimitTest.php`	Por pantalla (NAT-safe)	Cuotas independientes por token; HTTP 429 respetuoso	**Aprobado**
Carga Masiva (50+ Nodos)	Servidor Central / Nginx	p95 < 200 ms	Plan formal diseñado para ejecución en Hito 3	**Pendiente Declarada**

7. Registro de Mejoras al Producto Derivadas de las Pruebas

En estricta concordancia con los indicadores de calidad y metodología de Duoc UC, las pruebas de software en ACE no se conciben como un filtro pasivo al final del desarrollo, sino como un **mecanismo activo de perfeccionamiento continuo del producto**.

A lo largo del desarrollo e integración del corte vertical del Hito 2 (E-003), la ejecución iterativa de pruebas unitarias, de integración, de seguridad y de desconexión reveló defectos técnicos, condiciones de carrera y oportunidades de optimización que fueron remediados de manera inmediata y verificable. A continuación se documentan las diez mejoras principales incorporadas al software como resultado directo de este ciclo empírico:

7.1 Detalle de Mejoras Implementadas

1. Configuración de Directorio Temporal de Subidas en PHP (R-028)

- **Defecto descubierto en pruebas:** Al intentar subir archivos GIF de alta resolución (7,8 MB) en el panel administrativo de Filament, Livewire arrojaba un error HTTP `422 Unprocessable Content` (`UPLOAD_ERR_CANT_WRITE`) debido a que la directiva `upload_tmp_dir` no estaba configurada en PHP, intentando escribir en el directorio restringido del sistema `C:\WINDOWS`.
- **Mejora aplicada al entorno y producto:** Se parametrizaron `upload_tmp_dir = "D:/laragon/tmp"` y `sys_temp_dir = "D:/laragon/tmp"` en `php.ini`. Esto no solo desbloqueó la carga de archivos pesados, sino que consolidó la política estricta de aislamiento de almacenamiento del proyecto, garantizando que ninguna operación temporal toque el disco del sistema operativo.

2. Manejo Selectivo de Fallas de Red en el Proceso Principal de Electron (`e8c2d72` / R-028)

- **Defecto descubierto en pruebas:** Durante cancelaciones o cierres abruptos de conexión por parte del servidor web local mientras la escritura a disco pausaba el stream, se detonaba la excepción no capturada `assert(!this.paused) at Parser.finish` del parser `undici` de Node.js, desplegando un cuadro de error del sistema operativo en el quiosco.
- **Mejora aplicada al producto:** Tras rechazar una solución preliminar que silenciaba todas las excepciones globalmente (`5907ce7`), se implementó un filtro de excepciones selectivo en `player/src/electron/policy.js`

(commit `e8c2d72`). El manejador tolera de forma estricta y exclusiva la anomalía documentada de `undici`; cualquier otra falla no capturada se registra y gatilla un reinicio seguro del runtime desde la caché local física sin exponer diálogos de error al público.

3. Conmutación Gapless y Erradicación de Cuadros Negros (`7c1da71` / R-028)

- **Defecto descubierto en pruebas:** Las primeras conmutaciones entre ítems de una lista de reproducción producían parpadeos molestos donde la pantalla institucional `#state` (*"ACE PLAYER"*) quedaba visible hasta por 25 fotogramas (400 ms) antes de renderizar el siguiente video.
- **Mejora aplicada al producto:** Se rediseñó el ciclo de vida de renderizado en `player/src/renderer/player.js` (commits `7f36f2e` y `7c1da71`). Se estableció una política de retención activa: el medio saliente permanece visible y congelado en pantalla hasta que el nuevo recurso entrante emite la señal de cuadro listo (`player:frame`), garantizando **0 cuadros negros institucionales** en conmutaciones complejas de imagen a video y viceversa, certificado mediante 3 pruebas de regresión en `player/tests/renderer.test.js`.

4. Asistente Portátil de Configuración en Red Local (`520c256` / R-028)

- **Defecto descubierto en pruebas:** La necesidad de aprovisionar manualmente direcciones IP y Bearer Tokens editando archivos de configuración en terminales remotas durante las pruebas en red Wi-Fi resultaba lenta y propensa a inconsistencias tipográficas.
- **Mejora aplicada al producto:** Se desarrolló el script interactivo y portable `Iniciar_Player.bat` (commit `520c256`) en la raíz del repositorio. El script guía de forma interactiva al operador para ingresar la URL del backend y el token de pantalla, valida la conectividad y lanza el runtime en pantalla completa de forma estandarizada.

5. Blindaje contra Manipulación de Rutas y Path Traversal (`db59404` / T-043 / R-031)

- **Defecto descubierto en pruebas:** La auditoría de seguridad reveló que el formulario de carga aceptaba rutas enviadas por el cliente, abriendo la posibilidad de inyectar rutas arbitrarias (`../`) para leer, sobrescribir o eliminar archivos del servidor.
- **Mejora aplicada al producto:** Se incorporó el método `preventFilePathTampering()` en `MediaForm` y se restringió la ingesta en `MediaIngestor` exclusivamente a archivos contenidos directamente en el staging temporal `media/incoming` con formato ASCII canónico, auditado con 38 pruebas focales y 183 aserciones en `MediaIngestorTest.php` y `MediaResourceTest.php`.

6. Aislamiento Territorial Multi-Tenant en Selectores Reactivos (`db59404` / T-044 / R-032)

- **Defecto descubierto en pruebas:** Mediante manipulación de identificadores foráneos en peticiones reactivas de Livewire, un operador asignado a una locación podía llegar a visualizar nombres de medios, pantallas o sectores pertenecientes a otra sucursal.
- **Mejora aplicada al producto:** Se integró de manera transversal el alcance de autorización `LocationScope` en `PlaylistForm::libraryOf()` y `ScheduleForm::locationOf()`. Al inyectar un ID no asignado, el formulario retorna opciones vacías, bloquea la resolución del nombre privado y rechaza la persistencia, blindado con 29 pruebas y 194 aserciones en `PlaylistResourceTest.php` y `ScheduleResourceTest.php`.

7. Directiva Estricta de Privacidad HTTP en Medios (`db59404` / T-045 / R-033)

- **Defecto descubierto en pruebas:** Las respuestas de streaming servidas por `BinaryFileResponse` utilizaban por defecto cabeceras de caché pública, arriesgando que intermediarios HTTP o servidores proxy almacenaran y reutilizaran medios corporativos sin validar credenciales.

- **Mejora aplicada al producto:** Se configuró explícitamente `setCache(['private' => true, 'no_store' => true])` (RFC 9111) en `MediaController`. Mediante pruebas con `Symfony HttpCache` en `MediaDownloadTest.php`, se garantizó que cualquier petición posterior sin token o con token revocado reciba códigos 401/404 sin entrega de bytes cacheados.

8. Serialización Pesimista de Credenciales de Pantalla (`db59404` / T-046 / R-034)

- **Defecto descubierto en pruebas:** Solicitudes concurrentes de aprovisionamiento de tokens para una misma pantalla provocaban una condición de carrera que permitía la coexistencia de dos credenciales válidas en MariaDB.
- **Mejora aplicada al producto:** En `ScreenTokenIssuer`, la emisión se blindó con una transacción que adquiere un bloqueo pesimista de fila (`SELECT ... FOR UPDATE`) sobre la pantalla y ejecuta la revocación atómica por clave primaria (`DELETE WHERE id = ?`). El comportamiento fue certificado mediante el arnés multi-proceso `TokenIssuanceRace.php` en `ScreenTokenIssuerTest.php`.

9. Recuperación en Caliente de la Caché sin Parpadeos (`6c24865` / T-047 / R-035)

- **Defecto descubierto en pruebas:** La eliminación accidental o corrupción física de un archivo multimedia en disco durante la ejecución del Player dejaba al Viewport en blanco o exigía reiniciar manualmente toda la aplicación.
- **Mejora aplicada al producto:** Se incorporó en `CacheStore` y `SyncService` un mecanismo de detección proactiva periódica (cada 30 segundos) y observadores de eventos en el protocolo local (`MediaResources`). El Player reporta el diagnóstico, fuerza la resincronización de la misma versión de manifiesto y re-descarga el archivo dañado sin parpadeos visuales ni reinicio del proceso Electron, verificado con 14 casos en `live-cache-recovery.test.js`.

10. Erradicación Preventiva de Vulnerabilidades en Dependencias (`ed41e4a` / T-048 / R-036)

- **Defecto descubierto en pruebas:** El escaneo estático automatizado (`npm audit`) detectó vulnerabilidades de severidad en paquetes de soporte de desarrollo (`shell-quote 1.9.0` y `source-map-js 1.2.1`).
- **Mejora aplicada al producto:** Se actualizó puntualmente `concurrently` a `10.0.6` (incorporando `shell-quote 1.12.0` para neutralizar texto de comandos inseguro) y `source-map-js` a `1.2.2`, logrando un informe certificado de **0 vulnerabilidades** en producción en auditorías de `npm` y `composer audit`.

7.2 Matriz de Trazabilidad: Hallazgos Empíricos → Mejoras de Calidad

ID	Defecto / Hallazgo Empírico	Componente Afectado	Solución / Mejora Implementada	Commit / Archivo Canónico	Beneficio de Calidad Aportado
M-01	Error HTTP 422 en subidas grandes (`UPLOAD_ERR_CANT_WRITE`)	Backend PHP / Filament	Parametrización de `upload_tmp_dir = "D:/laragon/tmp"` en `php.ini`	R-028	Permite ingesta de archivos pesados y aísla el disco C:.
M-02	Excepción `assert(!this.paused)` de `undici` en Electron	Runtime Player / Red	Filtro selectivo en `policy.js` y reinicio automático desde caché	Commit `e8c2d72` / `main.js`	Tolerancia a desconexiones de socket sin bloquear quiosco.
M-03	Parpadeo de hasta 25 cuadros negros (`#state`) en conmutación	Viewport Renderer (HTML5)	Retención del medio saliente hasta renderizado del entrante	Commit `7c1da71` / `player.js`	**0 cuadros negros** (emisión publicitaria gapless continua).
M-04	Dificultad en configuración manual de terminales LAN	Operación y Despliegue	Script interactivo y portable `Iniciar_Player.bat`	Commit `520c256` / Raíz	Aprovisionamiento rápido y sin errores tipográficos.

ID	Defecto / Hallazgo Empírico	Componente Afectado	Solución / Mejora Implementada	Commit / Archivo Canónico	Beneficio de Calidad Aportado
M-05	Riesgo de inyección de rutas y Path Traversal en subidas	Ingesta de Medios	`preventFilePathTampering()` y restricción a `media/incoming`	Commit `db59404` / R-031	Blindaje absoluto del sistema de archivos del servidor.
M-06	Filtración de nombres/IDs privados entre locaciones	Livewire / Panel	Aislamiento territorial estricto con `LocationScope`	Commit `db59404` / R-032	Garantía de aislamiento multi-tenant corporativo.
M-07	Riesgo de almacenamiento de medios en proxies públicos	API Streaming HTTP	Cabecera `Cache-Control: private, no-store` (RFC 9111)	Commit `db59404` / R-033	Imposibilidad de que intermediarios sirvan medios sin token.
M-08	Carrera concurrente en emisión de tokens Sanctum	API de Pantallas / SQL	Transacción MariaDB con `SELECT ... FOR UPDATE`	Commit `db59404` / R-034	Coexistencia de exactamente un único token activo.
M-09	Bloqueo por eliminación o daño de archivo en caché local	Sincronización y Caché	Detección periódica y re-descarga transparente en caliente	Commit `6c24865` / R-035	Auto-recuperación de archivos sin parpadeos visuales.
M-10	Advisories de seguridad en dependencias de desarrollo	Dependencias npm	Actualización a `concurrently 10.0.6` y `source-map-js 1.2.2`	Commit `ed41e4a` / R-036	**0 vulnerabilidades** en auditorías de producción.

---

8. Evidencias Empíricas de Ejecución sobre Git Main

En cumplimiento riguroso del criterio de aceptación de la tarea T-057, todos los resultados presentados en este documento provienen de **ejecuciones verificadas sobre la rama principal (`main`) en el commit de corte `c328e07`**. Los tiempos sin medición formal se declaran como pendientes en la sección 6.

8.1 Línea Base de Software Auditada

- **Repositorio de código fuente:** `tomiloki/ace-plataforma` (privado).
- **Rama de referencia:** `main`.
- **Commit de corte auditado:** `c328e07` (*Merge pull request #9: Integrar hardening de seguridad, prevención de path traversal y dependencias*).
- **Fecha de consolidación y verificación:** 2026-10-12 (integración en `main`) y 2026-10-15 (consolidación formal de calidad).
- **Estado vigente de `main`:** después del corte se integraron los PR 10 y 11 de `ace-plataforma`. El commit `2350b15` aprobó Pint, pruebas del backend, pruebas del Player e integración de API y Player en la ejecución 38836701565 de GitHub Actions. Los conteos de este documento corresponden a `c328e07`.
- **Entorno de ejecución de pruebas:**
  - **Backend:** PHP 8.3.16 sobre Laravel 13, motor MariaDB 11.4 en contenedor Docker / entorno local `ace_testing`.
  - **Player:** Node.js v22.23.1 con runner nativo `node:test`, Chromium embebido en Electron v33.

8.2 Resultados Consolidados de la Suite de Backend

La suite de pruebas automatizadas del backend valida los modelos de datos, reglas de integridad referencial, controladores de la API REST del Player, recursos de administración en Filament y los seis frentes de seguridad:

```
+-------------------------------------------------------------------------------+
|              EJECUCIÓN DE SUITE BACKEND (PEST ^5.2 / PHPUNIT 11)              |
+-------------------------------------------------------------------------------+
| Comando: php vendor/bin/pest --colors=never                                   |
| Pruebas totales ejecutadas: 249 aprobadas (100%)                              |
| Aserciones verificadas:     1.027 aserciones                                  |
| Fallos detectados:          0 fallos                                          |
| Pruebas omitidas:           0 omitidas                                        |
| Tiempo de ejecución:        ~18,4 segundos                                    |
+-------------------------------------------------------------------------------+
| Verificación de Estilo:     Laravel Pint 100% aprobado (php vendor/bin/pint --test)|
| Auditoría de Seguridad:     composer audit -> 0 vulnerabilidades / 0 abandonados  |
+-------------------------------------------------------------------------------+
```

8.3 Resultados Consolidados de la Suite del Player Core

El núcleo del reproductor en Node.js se evalúa mediante pruebas asíncronas con temporizadores controlados, mocks de red y almacenamiento transaccional en disco:

```
+-------------------------------------------------------------------------------+
|              EJECUCIÓN DE SUITE PLAYER CORE (NODE.JS 22 / NODE:TEST)          |
+-------------------------------------------------------------------------------+
| Comando: npm --prefix player test                                             |
| Pruebas estándar ejecutadas: 235 aprobadas (100%)                             |
| Fallos detectados:           0 fallos                                         |
| Pruebas opt-in omitidas:     2 omitidas (requieren servicios externos levantados)|
| Resultados totales:          237 suites registradas                           |
+-------------------------------------------------------------------------------+
| Cobertura de Código Sinc.:   97,66% líneas | 93,24% ramas | 98,80% funciones      |
| Auditoría de Dependencias:   npm --prefix player audit -> 0 vulnerabilidades      |
+-------------------------------------------------------------------------------+
```

8.4 Verificación de las Suites de Integración Opt-In

Las dos pruebas de integración del Player que dependen de infraestructura en ejecución fueron validadas de forma independiente sobre el entorno de pruebas, alcanzando un 100% de éxito:

1. **Integración Player ↔ Laravel (`player/tests/laravel-sync.test.js`):**
   - **Invocación:** `ACE_PLAYER_LARAVEL_TEST=1 node --test player/tests/laravel-sync.test.js`.
   - **Resultado: 2 de 2 resultados aprobados** (0 omitidos).
   - **Garantía comprobada:** Siembra y purga de base de datos `ace_testing`, soporte de descargas parciales HTTP 206 (`Range`), persistencia atómica en disco y continuidad ante pérdida de red y token revocado.
2. **Integración Player ↔ Runtime Electron (`player/tests/electron-integration.test.js`):**
   - **Invocación:** `ACE_PLAYER_ELECTRON_TEST=1 node --test player/tests/electron-integration.test.js`.
   - **Resultado: 1 de 1 resultado aprobado** (0 omitidos).
   - **Garantía comprobada:** Generación de medios FFmpeg (PNG, JPG, GIF a 6 fps, MP4 H.264), decodificación acelerada en Chromium, conmutación gapless, generación de capturas en `output/playwright/` y salida limpia `ELECTRON_PASS`.

8.5 Certificación de Integración Continua (CI / GitHub Actions)

Para asegurar la reproducibilidad independiente fuera de las estaciones de trabajo de los desarrolladores, el repositorio central ejecuta un pipeline automatizado de Integración Continua en cada fusión a `main`:

- **Enlace a la ejecución de GitHub Actions verificada:** 👉 **GitHub Actions Run #38319434943 — Commit c328e07**
- **Trabajos ejecutados y certificados en el pipeline:**
  1. `Backend Tests`: Setup PHP 8.3, MariaDB service, migraciones y ejecución completa de Pest (**Aprobado**).
  2. `Code Style`: Laravel Pint sobre el código base (**Aprobado**).
  3. `Player Core Tests`: Setup Node.js 22 y ejecución de suite estándar `node:test` (**Aprobado**).
  4. `Player & API Integration`: Integración entre arnés PHP y sincronizador (**Aprobado**).
  5. `GitGuardian Security Scan`: Escaneo automatizado de secretos y credenciales en el historial (**Aprobado**).

8.6 Declaración Formal de Pruebas Pendientes

En estricta honestidad técnica y cumplimiento metodológico, se declaran formalmente las pruebas que no formaron parte del alcance del Hito 2 y quedan agendadas con sus tareas correspondientes:

Prueba Pendiente	Justificación Metodológica	Tarea Asignada
Homologación en Hardware NUC Físico	El corte vertical se certificó en PCs de desarrollo con Windows 11 y red Wi-Fi; la validación final sobre el hardware Intel NUC dedicado se ejecutará en la etapa de despliegue físico.	**T-037 - Preparar el NUC y la red local para las pruebas**
Pruebas de Carga y Concurrencia Masiva	La simulación de estrés de 50 a 100+ pantallas concurrentes enviando latidos a través de k6/Artillery está planificada para la fase de escalabilidad.	**Planificada para Hito 3**
Tratamiento de Medianoche en Horarios	La validación horaria actual bloquea franjas invertidas en el mismo día (`end_time > start_time`); la regla de negocio para franjas que cruzan medianoche permanece en diseño.	**T-032 - Definir el tratamiento de las franjas que cruzan medianoche**

8.7 Conclusión del Aseguramiento de la Calidad para el Hito 2

El plan de pruebas, las suites automatizadas y las evidencias empíricas consolidadas en este documento demuestran fehacientemente que el proyecto ACE cumple con los más altos estándares de ingeniería de software requeridos por la metodología de Duoc UC para la evaluación de avance de la Fase 2:

1. **Corte Vertical Plenamente Funcional:** El recorrido completo desde la administración territorial en el panel hasta la reproducción gapless en pantalla se encuentra respaldado por suites automatizadas y pruebas de red local.
2. **Resiliencia Offline Verificada:** Se comprobó empíricamente que la caída física de la red o del servidor no interrumpe la emisión continua desde la caché local, garantizando cero pantallas negras.
3. **Cero Regresiones de Seguridad:** Se auditaron y blindaron los seis vectores críticos de vulnerabilidad (Path Traversal, multi-tenant territorial, privacidad HTTP, carreras de tokens, integridad SHA-256 y dependencias limpias).
4. **Mejora Continua Demostrada:** Se trazaron diez optimizaciones técnicas de producto nacidas directamente del ciclo de testing.

Con esta base verificada sobre Git `main`, el sistema se declara formalmente **aprobado en calidad y listo para la presentación académica del Hito 2**.